

Laboratorio Guiado: Ataque de Padding Oracle en AES-CBC

Curso de Criptografía, NRC: 1984

Informe de Implementación

Integrante 1: Luis Cabarcas Romero
ID/Correo: lcabarcase@uninorte.edu.co

Integrante 2: Ashley Mercado Defort
ID/Correo: agmercado@uninorte.edu.co

1 Introducción

Este informe documenta la implementación del ataque de padding oracle contra AES-CBC con PKCS#7. El objetivo principal fue desarrollar un algoritmo capaz de recuperar texto plano únicamente utilizando la información booleana proporcionada por un oracle de padding vulnerable.

1.1 Contexto del Problema

El ataque se basa en la vulnerabilidad que surge cuando un sistema revela información sobre la validez del padding PKCS#7 durante el proceso de descifrado. Esta aparentemente pequeña filtración de información permite recuperar completamente el texto plano original mediante manipulación cuidadosa de los bloques de cifrado.

2 Enfoque de Implementación

2.1 Estrategia General

La implementación se basó en el principio del **invariante de sufijo**, procesando los bloques de derecha a izquierda (byte por byte) para forzar patrones de padding PKCS#7 válidos.

El enfoque consiste en manipular el bloque anterior (C'_{i-1}) para controlar el resultado del descifrado del bloque actual (C_i). Para cada posición de padding p (de 1 a 16 bytes), se mantiene un sufijo válido y se busca exhaustivamente el byte que produzca un padding correcto, revelando así información sobre el estado intermedio $I = D_k(C_i)$.

2.2 Invariante de Sufijo

Para cada longitud de padding p (donde $1 \leq p \leq 16$), se mantiene el siguiente invariante:

$$C'_{i-1}[j] = I[j] \oplus p \quad \text{para todo } j > t \quad (1)$$

donde $t = 16 - p$ es el índice del byte activo, e $I[j]$ representa los valores del estado intermedio ya recuperados.

Este invariante garantiza que los bytes ya procesados formen un sufijo de padding válido, permitiendo concentrarse únicamente en el byte activo de cada iteración.

2.3 Procedimiento Detallado

El algoritmo implementado sigue estos pasos:

1. **Inicialización:** Para cada bloque C_i , inicializar arrays $I[16]$ y $M[16]$ para almacenar el estado intermedio y el texto plano respectivamente.
2. **Iteración por posición de padding:** Para $p = 1, 2, \dots, 16$:
 - Calcular índice activo: $t = 16 - p$
 - Crear una copia modificable del bloque anterior: `c_prev_mod = bytearray(C_prev)`
 - Preparar sufijo manteniendo el invariante: `c_prev_mod[j] = I[j] ^ p` para $j \in \{t + 1, \dots, 15\}$
 - Realizar barrido exhaustivo: probar valores $g \in [0, 255]$ para `c_prev_mod[t]`
 - Consultar oracle con payload de 2 bloques: `bytes(c_prev_mod) + C_curr`
 - Al encontrar padding válido: calcular $I[t] = g \oplus p$ y $M[t] = I[t] \oplus C_{prev}[t]$

3. Procesamiento completo del mensaje:

- Dividir ciphertext en bloques de 16 bytes
- Aplicar recuperación bloque por bloque usando el anterior como IV
- Concatenar bloques recuperados y aplicar `unpad()` PKCS#7

La función `recover_block()` implementa el ataque por bloque, mientras que `recover_message()` orquesta la recuperación completa del mensaje.

3 Métricas de Rendimiento

3.1 Número de Consultas al Oracle

Métrica	Valor Teórico	Valor Obtenido	Eficiencia
Consultas/byte (promedio)	128	133	96.2 %
Consultas/byte (peor caso)	256	171	149.7 %
Consultas/bloque (promedio)	2048	2128	96.2 %

Cuadro 1: Comparación de métricas teóricas vs implementación

3.2 Análisis de Resultados

Los resultados experimentales muestran que la implementación alcanza una eficiencia promedio del 96.2 % respecto al valor teórico esperado. Las consultas por bloque varían significativamente:

- **Rango observado:** 1512-2734 consultas por bloque
- **Promedio:** ~2128 consultas por bloque
- **Casos exitosos:** 6 de 8 mensajes de prueba

Las variaciones se deben a la naturaleza probabilística del ataque y las características específicas de cada mensaje. Los fallos observados (Tests 5 y 6) sugieren posibles casos edge con patrones específicos que requieren análisis adicional.

4 Evidencia de Ejecución

4.1 Caso de Prueba

```

1 # Servidor vulnerable
2 server = VulnerableServer()
3
4 # Mensaje de prueba
5 plaintext = b"Attack at dawn!!"
6 print(f"Mensaje original: {plaintext}")
7
8 # Cifrado
9 ciphertext = server.encrypt(plaintext)
10 print(f"Ciphertext: {ciphertext.hex()}")
11
12 # Ataque de recuperaci n

```

```
13 recovered = recover_message(server.decrypt, ciphertext)
14 print(f"Mensaje recuperado: {recovered}")
15
16 # Verificación
17 assert recovered == plaintext
18 print(" Ataque exitoso!")
```

Listing 1: Ejemplo de ejecución del ataque

4.2 Salida del Programa

```
--- Test 1: Mensaje original: b'Attack at dawn!!' ---
Ciphertext (hex): 328a809bff0b78c5b5089dabe3c8388108fddfc669d4e791...
Longitud del ciphertext: 48 bytes
Recuperando mensaje de 2 bloques...
Recuperando bloque 1/2...
    Bloque recuperado con 2566 consultas
Recuperando bloque 2/2...
    Bloque recuperado con 1804 consultas
Mensaje recuperado: b'Attack at dawn!!'
El mensaje fue correctamente recuperado.

--- Test 3: Mensaje original: b'Top secret message The eagle has landed.' ---
Ciphertext (hex): 1d90db9e9cc712fbe024b8b8f283bc1d5ae71cf86a2feb0a...
Recuperando mensaje de 3 bloques...
Recuperando bloque 1/3...
    Bloque recuperado con 2650 consultas
Recuperando bloque 2/3...
    Bloque recuperado con 1859 consultas
Recuperando bloque 3/3...
    Bloque recuperado con 2231 consultas
Mensaje recuperado: b'Top secret message The eagle has landed.'
El mensaje fue correctamente recuperado.
```

5 Contramedidas

5.1 Medidas de Protección

Para prevenir ataques de padding oracle, se recomiendan las siguientes contramedidas:

1. **Tiempo constante:** Implementar rutinas de descifrado que tomen tiempo constante independientemente de la validez del padding
2. **Manejo uniforme de errores:** No distinguir entre errores de padding y otros errores de descifrado en las respuestas
3. **Modos de cifrado autenticados:** Utilizar modos como AES-GCM que proporcionan integridad además de confidencialidad
4. **MAC-then-Encrypt:** Aplicar códigos de autenticación de mensajes antes del cifrado para detectar manipulaciones

5.2 Implicaciones de Seguridad

Este ataque demuestra la importancia crítica de no filtrar información lateral durante operaciones criptográficas. Una señal booleana aparentemente inofensiva (validez del padding) puede comprometer completamente la confidencialidad del sistema.

La vulnerabilidad surge porque el oráculo de padding permite al atacante distinguir entre diferentes tipos de errores durante el descifrado, proporcionando información suficiente para reconstruir el texto plano byte por byte mediante manipulación sistemática de los bloques de cifrado.

6 Conclusiones

La implementación del ataque de padding oracle ha demostrado exitosamente cómo una filtración aparentemente mínima de información (validez del padding) puede resultar en la recuperación completa del texto plano.

Los resultados experimentales confirman la viabilidad práctica del ataque, con una eficiencia promedio del 96.2 % respecto a las métricas teóricas. De 8 casos de prueba, 6 fueron exitosamente descifrados, con un rango de 1512-2734 consultas por bloque.